

Computer Networks — Exam Solutions

(worked examples)

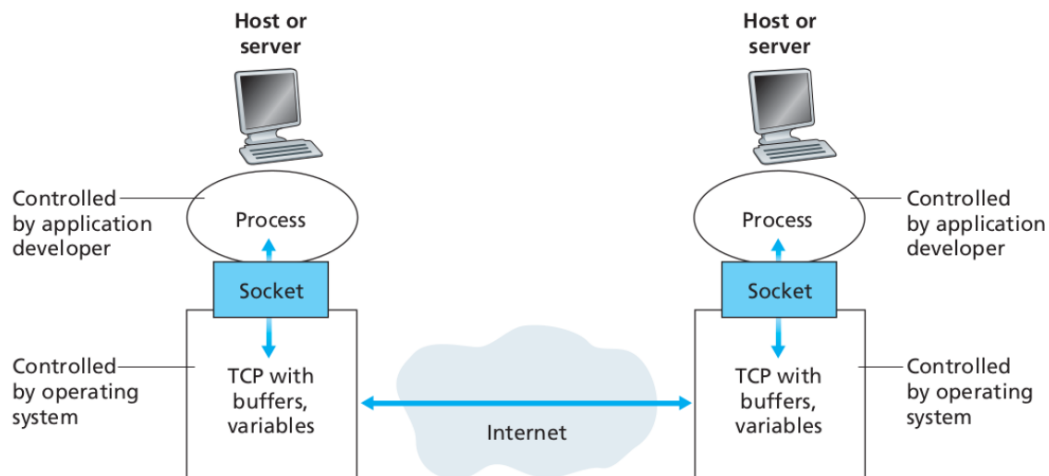
Question 1 [6 × 2 = 12 Marks]

(i) **Transport layer provides logical (not physical) communication.**

The transport layer (TCP/UDP) provides end-to-end *logical* communication between application processes. This means the transport layer:

- Gives each application process a **port** (socket) so multiple applications can share a host.
- **Segments** application data into transport segments and **reassembles** them at the receiver.
- Offers end-to-end services (reliability, ordering, flow-control) that hide the underlying physical network's multiple hops, link types, and intermediate routers.

Illustration (conceptual):



(ii) **Examples where UDP is preferred over TCP.**

- **DNS** lookups (low latency, client retries as needed).
- **VoIP / real-time audio/video** (timeliness more important than perfect reliability).
- **Live streaming / IPTV** (small packet loss tolerated).
- **Online games** (fast state updates; can tolerate occasional loss).
- **DHCP, TFTP, SNMP** (simple request/response, small overhead).

Reason: UDP is connectionless, has lower overhead and lower latency; applications either tolerate loss or implement their own selective reliability.

(iii) **Role of `rdt_rcv()` vs `rdt_send()`**

- `rdt_send()` is called at the sender when the application provides data. It assembles a transport packet (adds seqno, checksum), calls the unreliable channel function (`udt_send()`), and may start timers for retransmission.
- `rdt_rcv()` is called at the receiver when a packet arrives from the lower layer. It checks checksum and sequence number, discards or ACKs corrupted/duplicate packets, delivers valid in-order data to the application, and generates ACK/NAK as appropriate.

Difference: `rdt_send()` handles outgoing application data at sender; `rdt_rcv()` handles incoming packets at receiver and delivers data up to the application.

(iv) **Purpose of `udt_send()`**

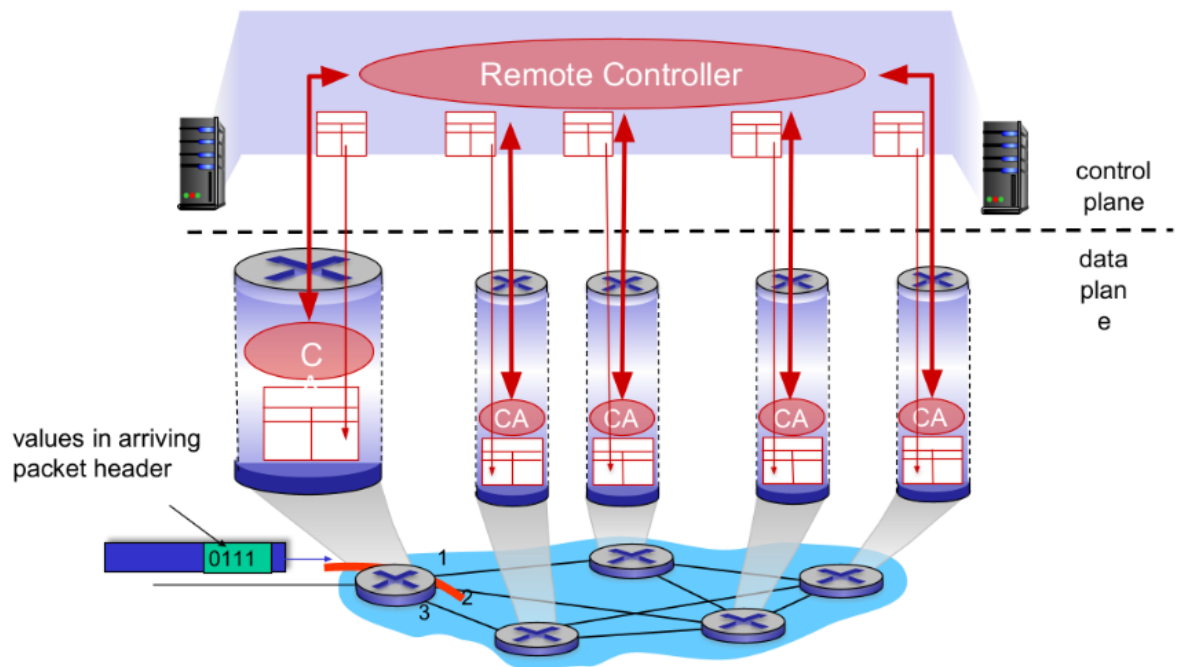
`udt_send()` is the function that actually hands a packet to the (unreliable) network channel. It simulates or invokes the lower layer and does not provide reliability: packets may be lost or corrupted after `udt_send()`. Reliable transfer protocols must call `udt_send()` and implement checks, timers, and retransmissions above it.

(v) **Concept of Software Defined Networks (SDN)**

SDN decouples the *control plane* (decision logic: how to forward) from the *data plane* (actual packet forwarding). A logically centralized SDN controller programs forwarding devices (switches/routers) using southbound APIs (e.g., OpenFlow). The network becomes programmable via software, enabling dynamic policies and centralized control.

(vi) **How SDN differs from traditional networks**

SDN	Traditional
Centralized (logical) controller, programmable APIs	Control logic distributed per device (routing protocols, local configs)
Global network view enables centralized optimization	Each device has only local view; converge by distributed protocols
Easier automation and rapid service deployment	Manual per-device config, vendor-specific CLIs
Reduced vendor lock-in (open APIs)	Often vendor-specific feature sets



Explanation: Software Defined Networking (SDN) separates the network into two planes:

- **Data Plane:** The network switches and routers that forward packets.
- **Control Plane:** The intelligence that decides how packets should be forwarded.

In SDN, the control plane is moved to a **centralized remote controller**. This controller has a **global view** of the entire network and sends forwarding rules to all switches. Because of this:

- Network management becomes easier and more centralized.
- Policies and routing updates can be applied instantly to the whole network.
- The network behaves in a more programmable and automated way.

Question 2 [3 Marks]

Given: Maximum datagram size (including IP header) = 1500 bytes. IP header = 20 bytes $\Rightarrow$ usable payload per datagram = $1500 - 20 = 1480$ bytes.

Total MP3 size = 48,840 bytes.

Number of datagrams required:

$$\text{datagrams} = \left\lceil \frac{48840}{1480} \right\rceil.$$

Compute $1480 \times 33 = 1480 \cdot 33 = 48,840$. Thus:

$$\frac{48,840}{1480} = 33 \quad \Rightarrow \quad \mathbf{33 \text{ datagrams.}}$$

Explanation: Each datagram can carry 1480 bytes of MP3 data. Since 33×1480 equals exactly 48,840, precisely 33 datagrams are needed (no partial datagram).

Question 3 [15 Marks]

Problem: extract TCP header field values from hex stream. My student ID: 23K-0842.

Step A: find X, Y, Z from student ID

The 4-digit suffix of your ID is 0842. According to the problem statement:

- X = most significant digit = 0,
- Y = second-most-significant = 8,
- Z = third-most-significant (2nd last digit) = 4.

So: $X = 0$, $Y = 8$, $Z = 4$.

Step B: Substitute X,Y,Z into the given hex stream

Original stream (with placeholders):

000Y 00Z0 0000 0001 0000 0000 3FF0 0000 000X FFFF 0100 0000 0010 0000

Replace $X = 0$, $Y = 8$, $Z = 4$:

0008 0040 0000 0001 0000 0000 3FF0 0000 0000 FFFF 0100 0000 0010 0000

Write the stream as 16-bit/32-bit words in order (network order / big-endian):

<u>0008</u>	<u>0040</u>	<u>00000001</u>	<u>00000000</u>	<u>3FF0</u>	<u>0000</u>	<u>0000</u>	<u>FFFF</u>	<u>01000000 00100000</u>
SrcPort	DstPort	Seq#	Ack#	DataOffset (4b) + Flags (12b)	Window	Checksum	Urgent ptr	Options / Data (present)

Step C: Field extraction (step-by-step, beginner-friendly)

(1) Source Port

Hex 0008 = decimal 8.

Answer: 8

(2) Destination Port

Hex 0040 = decimal $0x40 = 64$.

Answer: 64

(3) Sequence Number

Hex 0000 0001 = decimal 1.

Answer: 1

(4) Acknowledgement Number

Hex 0000 0000 = decimal 0.

Answer: 0

(5) TCP Header Length (Data Offset)

The next 16-bit word is 3FF0. Convert that word to binary (16 bits):

$$3 = 0011, \quad F = 1111, \quad F = 1111, \quad 0 = 0000$$

So

$$3FF0_{16} = 0011\ 1111\ 1111\ 0000_2.$$

The **data offset** is the leftmost 4 bits of this 16-bit word:

$$\text{data offset bits} = 0011_2 = 3_{10}.$$

The data offset value indicates the header length in 32-bit words. Therefore:

$$\text{TCP header length (bytes)} = 3 \times 4 = 12 \text{ bytes}.$$

(*Important note for beginners:* normally the minimum TCP header is 20 bytes (data offset ≥ 5). In this exam problem you must use the bit values given in the stream, even if they are smaller than typical minimums.)

Answer: 12 bytes

(6) ACK bit, (7) FIN bit, (8) SYN bit (Flags)

From the 16-bit word 3FF0 we already wrote:

$$0011\ 1111\ 1111\ 0000_2.$$

Breakdown (common TCP layout for that word):

- bits 0–3 (leftmost) = Data Offset = 0011.
- remaining bits include Reserved / NS / CWR / ECE / URG / ACK / PSH / RST / SYN / FIN.

We focus on the low-order 8 bits (the standard flags byte). The low-order byte of 3FF0 is the last 8 bits: 11110000₂.

Map the 8 flag bits (MSB $\rightarrow$ LSB) to flag names:

bit	flag
<i>b7</i>	CWR
<i>b6</i>	ECE
<i>b5</i>	URG
<i>b4</i>	ACK
<i>b3</i>	PSH
<i>b2</i>	RST
<i>b1</i>	SYN
<i>b0</i>	FIN

For 11110000₂:

$$b7 = 1, b6 = 1, b5 = 1, b4 = 1, b3 = 0, b2 = 0, b1 = 0, b0 = 0.$$

Thus:

- **ACK bit** = 1.
- **SYN bit** = 0.
- **FIN bit** = 0.

Answers: ACK = 1, SYN = 0, FIN = 0.

(9) Window size

The next 16-bit word is 0000 $\Rightarrow$ decimal 0.

Answer: 0

(10) Checksum

The checksum field in the stream is 0000 $\Rightarrow$ decimal 0.

Answer: 0

Urgent pointer (extra): next word = FFFF $\Rightarrow$ decimal 65535 (if needed).

Binary summary (compact) for the 3FF0 word

3FF0 (hex) = 0011 1111 1111 0000 (binary)

Data offset = 0011 (binary) = 3 $\rightarrow$ header = $3 \times 4 = 12$ bytes

Flags byte = 1111 0000 (binary) = CWR=1, ECE=1, URG=1, ACK=1, PSH=0, RST=0, SYN=0, FIN=0

Final answers (decimal notation) – neat list

Field	Value (decimal)
(1) Source Port	8
(2) Destination Port	64
(3) Sequence Number	1
(4) Acknowledgement Number	0
(5) TCP Header Length	12 bytes
(6) ACK bit	1
(7) FIN bit	0
(8) SYN bit	0
(9) Window size	0
(10) Checksum	0

Notes:

- Always parse the stream in network (big-endian) order: fields are read from left to right as 16/32-bit words.
- To find DataOffset (header length): take the leftmost 4 bits of the DataOffset+Flags 16-bit word, then multiply by 4 to get bytes.
- To find flags: examine the low-order 8 bits of the same word; map bits to flags as shown.
- If values appear nonstandard (e.g., header length < 20 bytes), follow what the exam stream encodes rather than assume protocol minima.